TCP: Transport Layer

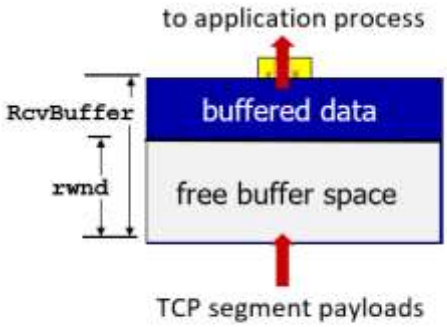
TCP Flow Control:

TCP provides a flow-control service to its applications to eliminate the possibility of the sender overflowing the receiver's buffer. Flow control is thus a speed-matching service—matching the rate at which the sender is sending against the rate at which the receiving application is reading.

TCP provides flow control by having the *sender* maintain a variable called the **receive window**. Informally, the receive window is used to give the sender an idea of how much free buffer space is available at the receiver. Because TCP is full-duplex, the sender at each side of the connection maintains a distinct receive window. Let's investigate the receive window in the context of a file transfer. Suppose that Host A is sending a large file to Host B over a TCP connection. Host B allocates a receive buffer to this connection; denote its size by **RcvBuffer**. From time to time, the application process in Host B reads from the buffer. Define the following variables:

- **LastByteRead:** the number of the last byte in the data stream read from the buffer by the application process in B
- **LastByteRcvd:** the number of the last byte in the data stream that has arrived from the network and has been placed in the receive buffer at B Because TCP is not permitted to overflow the allocated buffer, we must have **LastByteRcvd – LastByteRead <= RcvBuffer**
- The receive window, denoted rwnd is set to the amount of spare room in the buffer: **rwnd = RcvBuffer – [LastByteRcvd – LastByteRead]**
  Because the spare room changes with time, rwnd is dynamic.

How does the connection use the variable **rwnd** to provide the flow-control service? Host B tells Host A how much spare room it has in the receiving buffer by placing its current value of rwnd in the receive window field of every segment it sends to A. Initially, Host B sets rwnd = RcvBuffer. Note that to pull this off, Host B must keep track of several connection-specific variables.



TCP receiver-side buffering

Host A in turn keeps track of two variables, **LastByteSent** and **LastByteAcked**, which have obvious meanings. Note that the difference between these two variables, LastByteSent – LastByteAcked, is the amount of unacknowledged data that A has sent into the connection. By keeping the amount of unacknowledged data less than the value of rwnd, Host A is assured that it is not overflowing the receive buffer at Host B. Thus, Host A makes sure throughout the connection's life that

LastByteSent – LastByteAcked <= rwnd

There is one minor technical problem with this scheme. To see this, suppose Host B's receive buffer becomes full so that rwnd = 0. After advertising rwnd = 0 to Host A, also suppose that B has *nothing* to send to A. Now consider what happens. As the application process at B empties the buffer, TCP does not send new segments with new rwnd values to Host A; indeed, TCP sends a segment to Host A only if it has data to send or if it has an acknowledgment to send. Therefore, Host A is never informed that some space has opened up in Host B's receive buffer— Host A is blocked and can transmit no more data! To solve this problem, the TCP specification requires Host A to continue to send segments with one data byte when B's receive window is zero. These segments will be acknowledged by the receiver. Eventually the buffer will begin to empty and the acknowledgments will contain a nonzero rwnd value.